

Course: Portfolio Building studio course

Topic: Design a Simple Command Line Text Editor in C

Team Members:

1. Shrihari. H. Deai - R25EJ140

2. Souxabh. Toragal - R25EJ149

I) Data Structures Choice & Justification

typedef struct LineNode

char * text;

struct LineNode * next;

LineNode;

typedef struct

LineNode * head;

int line_count;

Document;

Justification:

- Insertion/Deletion Overhead: In a singly linked list, once traversed to position k.
- Dynamic Memory Utilisation
- Trade-off: sequential traversal is $O(N)$ to reach line k (vs $O(1)$ direct indexing as an array).

II) Visual Memory Diagram

doc (Document)

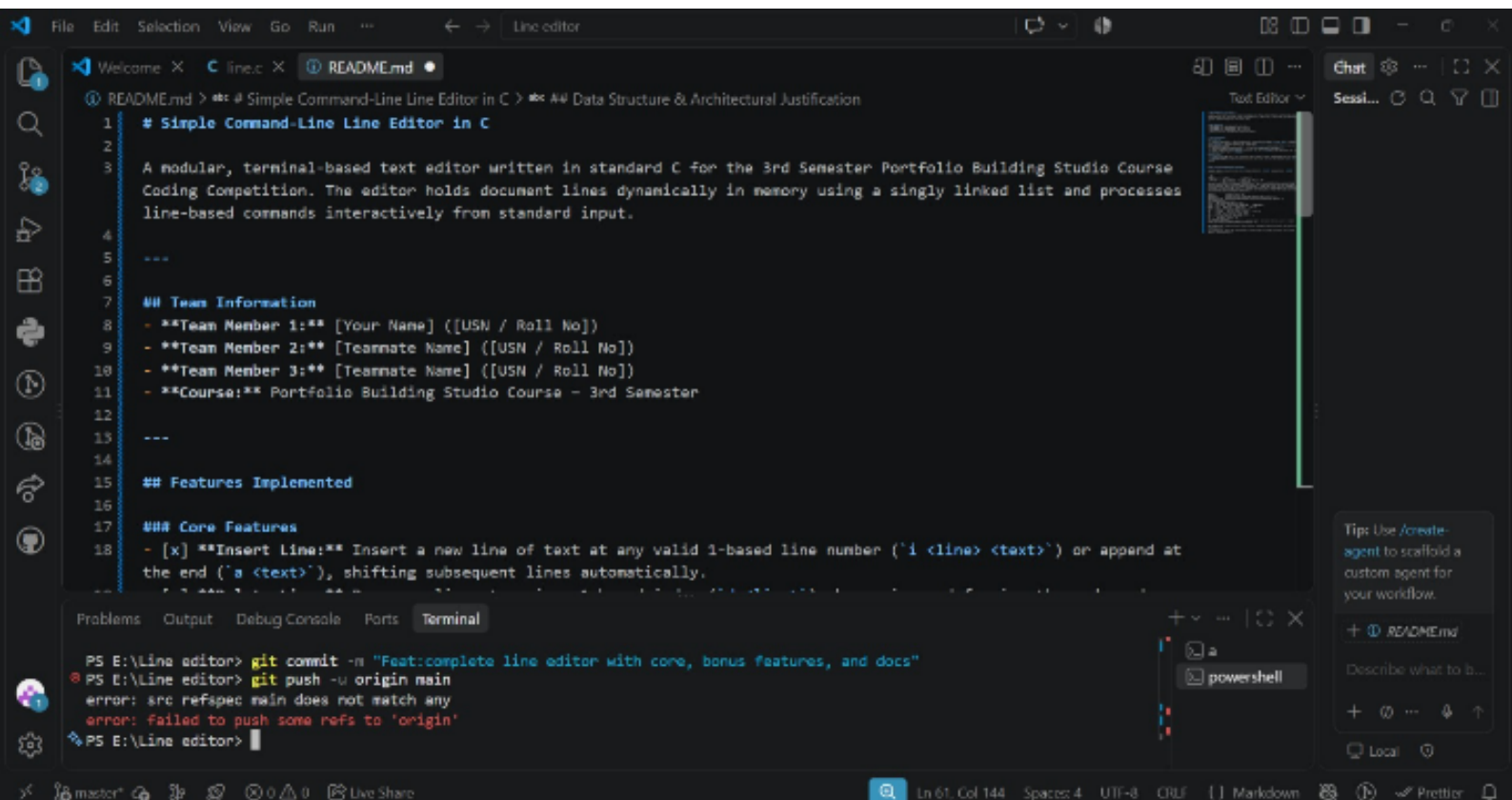
line_count: 2

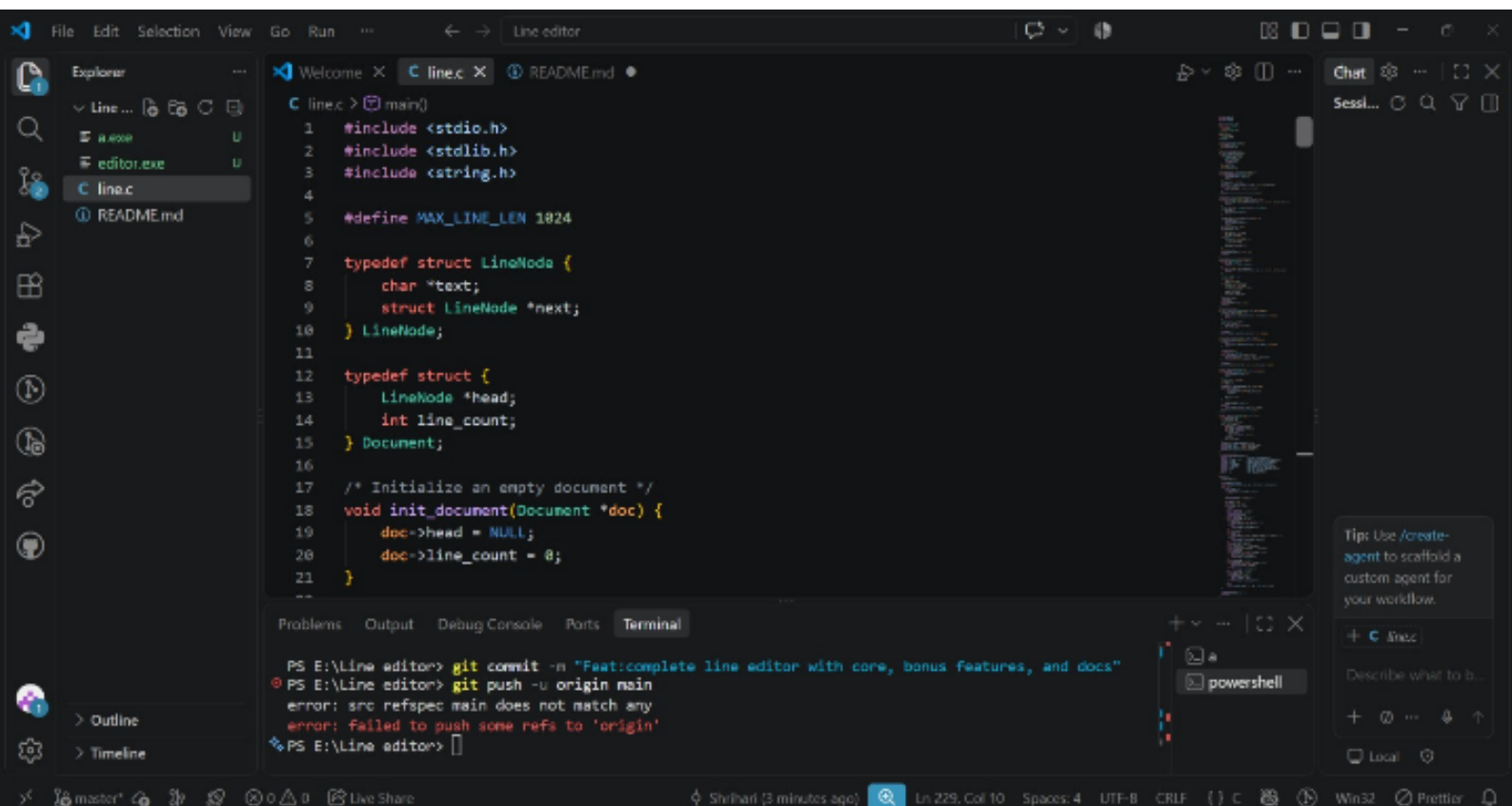
head → [Line Node 1] [Line Node 2]

| text: "Hello" | → | text: "World" | → NULL

| next: --- ↓ | next: NULL

[New Node]
| text: "Middle" |
| next: --- |





iii) Command Grammar & Interface

p - print the entire document

i <line> <text> - insert new text line position

a <text> - append text line

d <line> - Delete line at position

s <filename> - Write & save document buffer to disk

l <filename> - Load document lines from text file

f <query> - Search document lines

stat - Display metrics

q - Free all allocated memory blocks & exit.

iv) Edge cases Handled.

- Empty Document: Checking doc $\rightarrow$ head = NULL during display, search, & delete calls
- Deleting single remaining line: When line-count == 1 of line. It is removed, doc $\rightarrow$ head becomes NULL
- Out of Bounds Position
- Inserting at Extremities: validating pointer for pos == 1 vs pos == line-count + 1
- memory leak prevention: calling free(target $\rightarrow$ text) before free(target)

```
void display_document (const Document *doc) {
    if (doc->head == NULL) {
        printf ("[Document is empty]\n");
        return;
    }
    ListNode *curr = doc->head;
    int index = 1;
    printf ("\n--- Document Contents (%d lines) ---\n", doc->line_count);
    while (curr != NULL) {
        printf ("%4d | %s\n", index++, curr->text);
        curr = curr->next;
    }
    printf ("-----\n\n");
}

int insert_line (Document *doc, int pos, const char *text) {
    if (pos < 1 || pos > doc->line_count + 1) {
        printf ("Error: Invalid position. Valid range is 1 to %d.\n",
            doc->line_count + 1);
        return 0;
    }
    ListNode *new_node = (ListNode *) malloc (sizeof (ListNode));
    if (!new_node) {
        perror ("Allocation failed");
        return 0;
    }
    new_node->text = (char *) malloc (strlen (text) + 1);
    if (!new_node->text) {
        free (new_node);
        perror ("Allocation failed");
        return 0;
    }
    strcpy (new_node->text, text);
    new_node->next = NULL;
    if (pos == 1) {
        new_node->next = doc->head;
        doc->head = new_node;
    } else {
        ListNode *curr = doc->head;
        for (int i = 1; i < pos - 1; i++) {
            curr = curr->next;
        }
        new_node->next = curr->next;
        curr->next = new_node;
    }
    doc->line_count++;
    printf ("Line inserted at %d.\n", pos);
    return 1;
}
```

Top Private University in Bang...My Semester Courses | RPWline-editor-competition.pdfWhatsAppshriharidesai209-commits/line

github.com/shriharidesai209-commits/line-editor-of-file

shriharidesai209-commits / Line-editor-of-file

Type to search

CodeIssuesPull requestsAgentsActionsProjectsWikiSecurity and qualityInsightsSettings

Line-editor-of-file (Public)

PinWatch 0Fork 0Star 0

main1 Branch0 Tags

Go to file

Add file

Code

shriharidesai209-commits docs: add comprehensive README with design justification, build steps... 675b7d1 now 3 Commits

README.mddocs: add comprehensive README with design justification, ...now

line.cfeat: complete line editor with core, bonus features, and docu...3 minutes ago

editor.cfeat: complete line editor with core, bonus features, and docu...3 minutes ago

line.cfeat: complete line editor with core, bonus features, and docu...8 minutes ago

README

Simple Command-Line Line Editor in C

A modular, terminal-based text editor written in standard C for the 3rd Semester Portfolio Building Studio Course Coding Competition. The editor holds document lines dynamically in memory using a singly linked list and processes line-based commands interactively from standard input.

About

A minimal, terminal-based line text editor in C built with dynamic singly linked lists, supporting line insertion, deletion, file I/O, regex-free search, and document statistics.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

25°CPartly sunny

Search

10:18 AM24-09-2025